

Oops



CONTENT

01

Introduction

02

Nodes

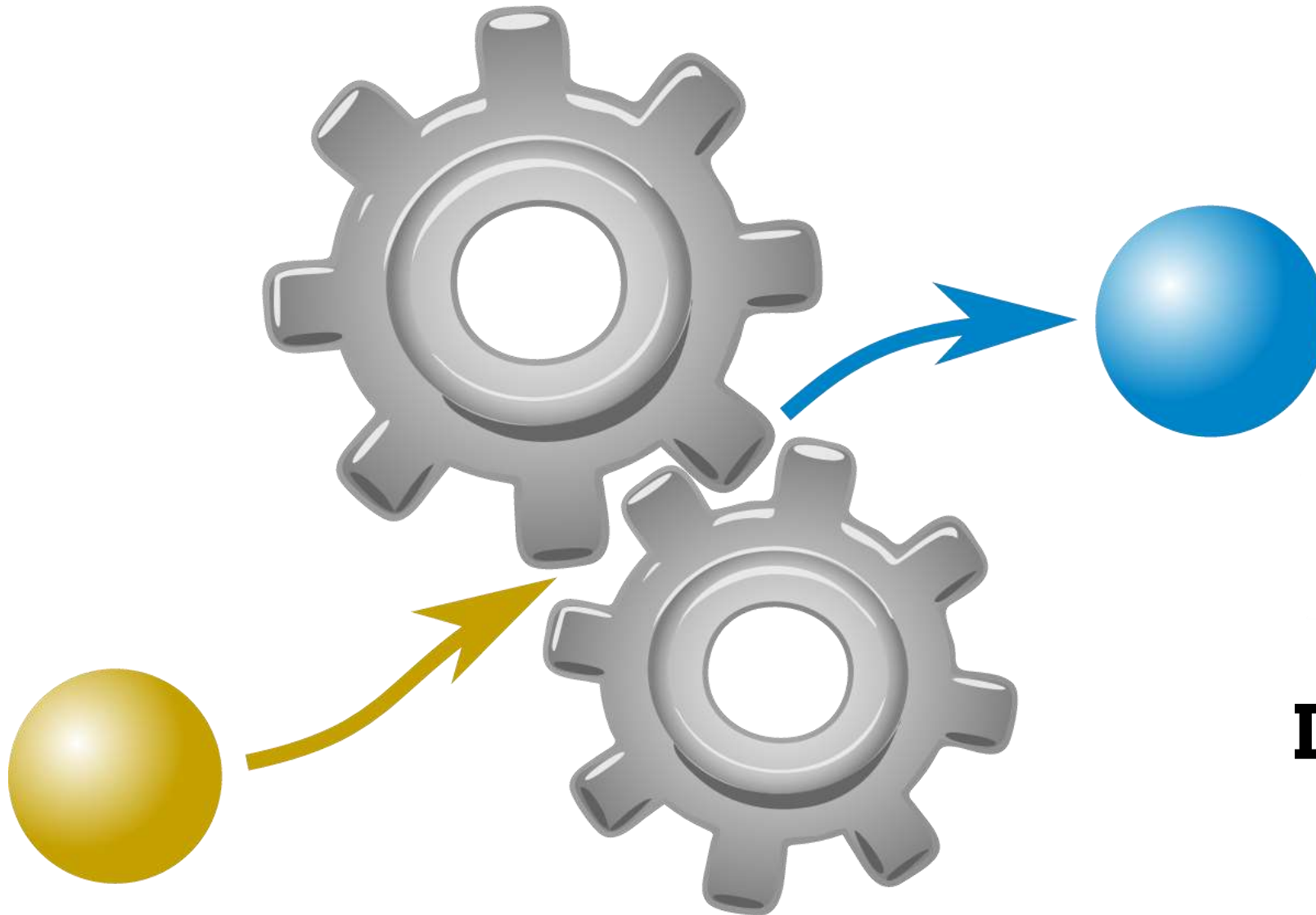
03

Selectors

04

Event





01

Introduction

OOPS



➤ **OOPS** stands for **Object-Oriented Programming System**.

- It is a **programming paradigm** (style) where software is designed around **objects** instead of just functions and logic.

Meaning :

- ❑ **Object-Oriented** → Based on **Objects** (real-world entities).
- ❑ **Programming** → Way to write code using objects.
- ❑ **System** → A structured, organized way to manage code.

➤ JavaScript is **prototype-based**, but from **ES6** onwards it supports **class-based OOPS** as well.



Introduction

Step 1: Basics of Class and Methods :

❖ What is a Class?

- A **class** is a **blueprint** to create objects.
- Introduced in **ES6** for cleaner OOPS syntax.

Example

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  greet() {  
    console.log(`Hello, my name is ${this.name}`);  
  }  
}  
  
const p = new Person("Nitish", 25);  
p.greet(); // Hello, my name is Nitish
```



Introduction

Step 2: Encapsulation (Grouping Data + Methods)

- **Encapsulation** means **bundling data (properties)** and **methods (functions)** inside a **single unit (object or class)** and **restricting direct access** to some components of that object.
- ✓ Think of it like a box that hides the internal details but provides public methods to interact with it.

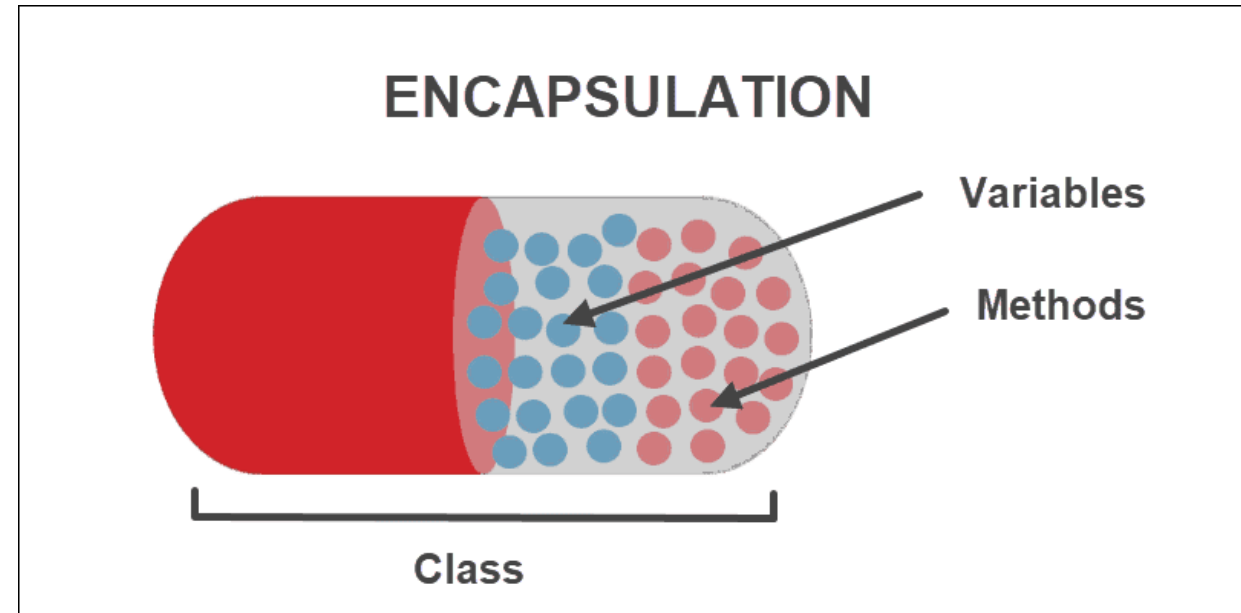
Why Use Encapsulation?

Benefit	Explanation
✓ Data Protection	Prevent direct modification of important data
✓ Cleaner Code	Code becomes modular, organized
✓ Better Control	Allows validation while accessing/updating data
✓ Hides Complexity	Internal logic remains hidden from outside



Without Encapsulation (Bad Practice)

```
class User {  
  
    constructor(name, age) {  
  
        this.name = name;  
  
        this.age = age;  
  
    }  
  
}
```



```
const user = new User("Nitish", 25);  
  
user.age = -100; // ✗ No control on invalid data  
  
console.log(user.age); // -100
```

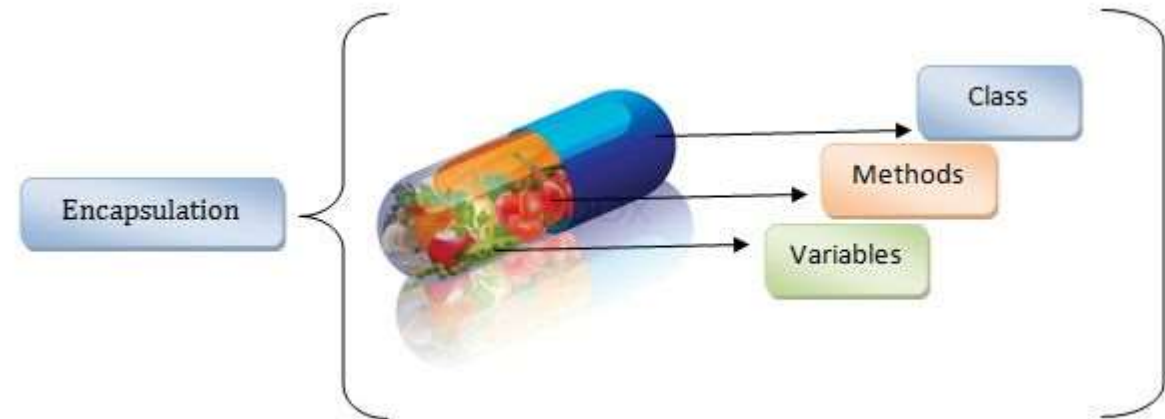


➤ With Encapsulation using Getter/Setter :

```
class User {
  constructor(name, age) {
    this.name = name;
    let _age = age; // private variable using closure

    this.getAge = function() {
      return _age;
    }

    this.setAge = function(value) {
      if(value > 0) {
        _age = value;
      } else {
        console.log("Invalid age");
      }
    }
  }
}
```



```
const user = new User("Nitish", 25);
console.log(user.getAge());    // 25
user.setAge(-5);
console.log(user.getAge());    // 25    // Invalid age
```

Internal `_age` is protected and accessed only via methods.



➤ Using Private Fields (#) in Modern JavaScript (ES2020):

Example

```
class BankAccount {  
  #balance;                                // private  
  field  
  constructor(initialBalance) {  
    this.#balance = initialBalance;  
  }  
  
  deposit(amount) {  
    if (amount > 0) this.#balance += amount;  
  }  
  
  getBalance() {  
    return this.#balance;  
  }  
}  
  
const acc = new BankAccount(1000);  
acc.deposit(500);  
console.log(acc.getBalance()); // 1500  
console.log(acc.#balance);    // ✗ Error:  
Private field
```

Private fields with # hide data from outside completely.



➤ Step 3: Abstraction (Hiding Internal Details)

- **Abstraction** means **hiding internal details** and exposing only the **necessary information** to the outside world.
- **Using private fields/methods to hide unnecessary details**



- You **drive a car** by using the **steering, brake, accelerator**, but you **don't need to know** how the **engine or brakes work internally**.
- The **complex engine logic is abstracted**.



Example

```
class Car {  
  #engineStatus = false;
```

```
  startEngine() {  
    this.#engineStatus = true;  
    console.log("Engine Started");  
  }  
}
```

```
const car = new Car();  
car.startEngine();  
console.log( car.#engineStatus );
```

// ✗ Error: Private field

The goal is to reduce complexity and make your code cleaner, safer, and more maintainable.



➤ Step 4: Inheritance (Code Reusability)

- **Inheritance** means **one class (child)** can **reuse the properties and methods** of another class (parent).
- **Purpose:** Reusability — avoid writing the same code again and again.

❖ Why Use Inheritance?

Benefit	Explanation
✓ Code Reusability	Child class reuses parent class code
✓ Organized Structure	Cleaner and maintainable project code
✓ Method Overriding	Customize behavior in child classes



❑ Inheritance in JavaScript (Modern Way with ES6 class)

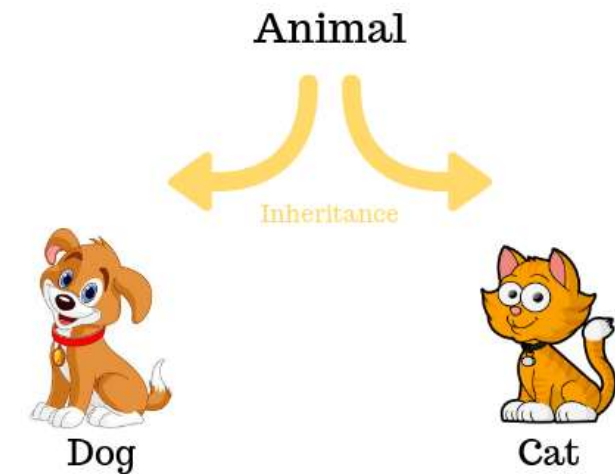
Example

```
// Parent Class
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound`);
  }
}

// Child Class
class Dog extends Animal {
  speak() {
    console.log(`${this.name} barks`);
  }
}

const dog = new Dog("Tommy");
dog.speak(); // Tommy barks
```



extends → inherits from parent
super() → calls parent class constructor



➤ Step 5: Polymorphism (Many Forms)

- **Polymorphism** means "**many forms**".
- In **OOPS**, polymorphism allows the **same method name** to behave **differently** depending on which object is calling it.

❖ Simple Real-Life Example:

- **Animal** → makes **generic sound**
 - **Dog** → **barks**
 - **Cat** → **meows**
- ✓ All animals have a `speak()` method — but it **behaves differently**.

Let's talk about two different method :

- **Method Overriding**
- **Method Overloading**



➤ What is Method Overriding?

- **Method Overriding** means when a **child class provides its own implementation** of a method that is already defined in its **parent class**.
- The **method name remains the same**, but the **behavior is changed** in the child class.

❖ Why Use Method Overriding?

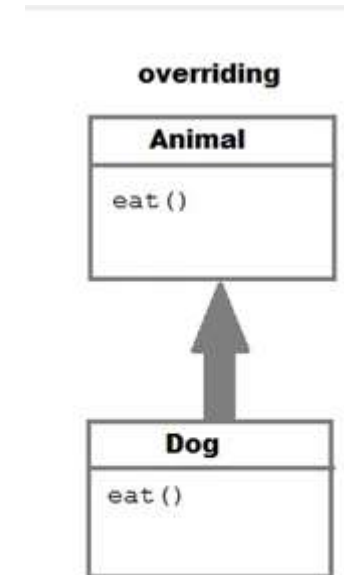
Benefit	✓ Explanation
Custom Behavior	Child class modifies the default logic
Polymorphism	Allows same method to behave differently based on the object
Reusability + Flexibility	Basic structure from parent; behavior from child



Example

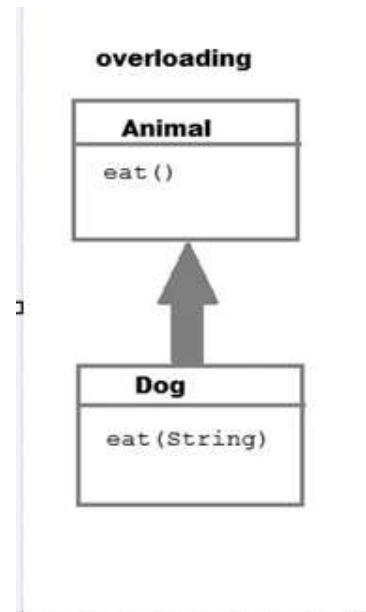
```
class Parent {  
    methodName() {  
        console.log("Parent Method");  
    }  
}  
  
class Child extends Parent {  
    methodName() {  
        console.log("Child Method (Overridden)");  
    }  
}
```

❖ Child overrides Parent's method.



➤ What is Method Overriding?

- **Method Overloading** means **defining multiple methods with the same name** but **different parameters** (number or type of arguments).



! Important Note for JavaScript:

JavaScript DOES NOT support method overloading in the traditional sense.

- If you declare multiple methods with the same name in a **class** or **object**, the **last method overwrites the previous ones**.



Example

```
class Animal {
  speak() {
    console.log("Animal speaks");
  }
}
```

```
class Dog extends Animal {
  speak() {
    console.log("Dog barks");
  }
}
```

```
class Cat extends Animal {
  speak() {
    console.log("Cat meows");
  }
}
```

```
const animals = [new Dog(), new Cat(), new Animal()];
animals.forEach(animal => animal.speak());
```

Dog barks
 Cat meows
 Animal speaks



Same method name → Different behavior → Polymorphism



Notes :

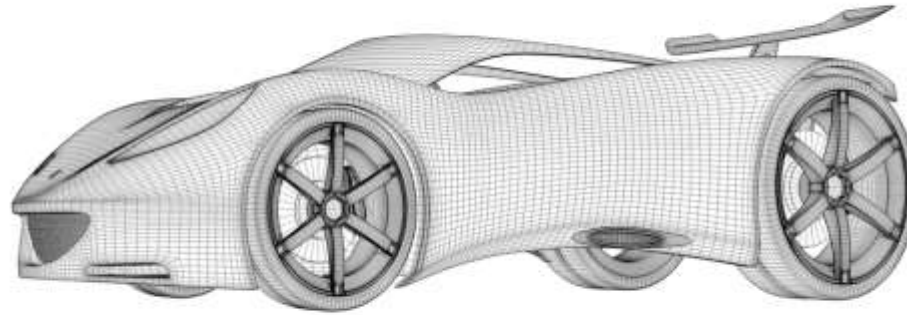


- ❖ Polymorphism happens during runtime (dynamic dispatch).
- ❖ JS focuses on method overriding (main way of achieving polymorphism).



Prototypes

❑ What is Prototype?



- Every function in JavaScript has a **prototype** property.
- Objects created via constructor functions inherit from the **prototype**.
- ✓ **Prototype** allows method sharing, reducing memory usage.



❖ **Example of Prototype with Constructor Function:**

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}  
  
// Method attached to prototype (shared across objects)  
Person.prototype.greet = function() {  
  console.log(`Hello, my name is ${this.name}`);  
};  
  
const p1 = new Person("Nitish", 25);  
const p2 = new Person("Kumar", 30);  
  
p1.greet(); // Hello, my name is Nitish  
p2.greet(); // Hello, my name is Kumar
```

greet() method is shared, not recreated for each object.

